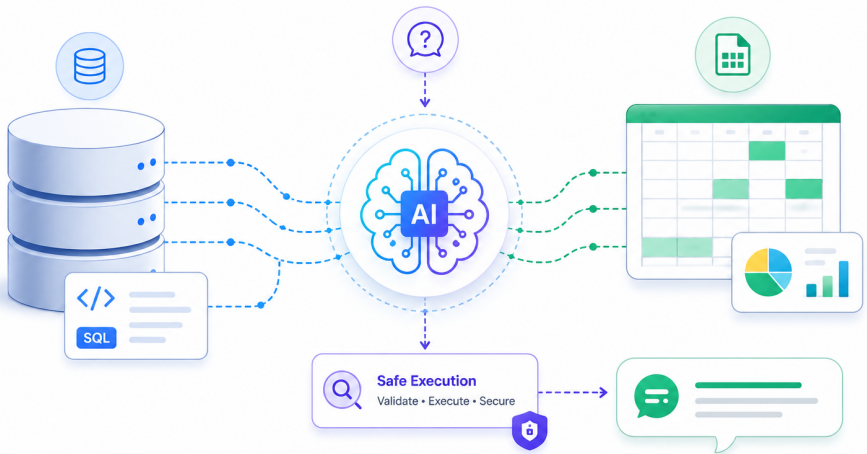


RAG Over Structured Data

When your knowledge base is a spreadsheet or a database, not a PDF.



Structured Data Breaks the Standard RAG Playbook

Why vector similarity fails on tabular and relational data

✖ **Not semantically similar:** Standard RAG chunks text and searches by semantic similarity, but a row in a spreadsheet or a database table isn't "semantically similar" to a question: it's *computed* from it (sums, filters, joins, averages).

📊 **Requires calculation:** "What was our highest-selling product last quarter?" isn't answered by finding a similar-sounding sentence; it requires actually running a query or calculation over the data.

🗃️ **Vector search loses structure:** Embedding every row as text and doing vector search loses the structure (columns, types, relationships) that makes the data useful in the first place.

THE CORE SHIFT:



Instead of retrieve-then-generate, structured RAG is closer to: **understand the question** → **generate a query/computation** → **execute it** → **explain the result**.



Where This Pattern Actually Gets Used

Key enterprise architectures relying on structured RAG

BUSINESS INTELLIGENCE CHATBOTS

"What were Q3 sales by region?" executed over a SQL data warehouse.

SPREADSHEET Q&A

Answering complex questions over CSVs or Excel exports without writing formulas manually.

CUSTOMER SUPPORT OVER TRANSACTIONAL DATA

"What's the status of order #4521?" queried directly against a live database.

FINANCIAL ANALYSIS ASSISTANTS

Aggregating and comparing financial numbers across large Excel workbooks.

HYBRID ASSISTANTS

Combining structured data (numbers, statuses) with unstructured docs (policies, contracts) in one answer.



AI Coach John



Follow

 Repost

What We Are Building **Today**

The end-to-end execution pipeline for structured data

1. Understand Question & Intent
2. Inspect & Understand Schema
3. Generate Query or Code
4. Execute Safely in Read-Only Sandbox
5. Validate & Handle Errors with Retries
6. Format Result as Natural Language Answer

1. TEXT-TO-SQL

For relational databases (PostgreSQL, SQLite, MySQL).

2. DATAFRAME AGENTS

For CSVs and Excel spreadsheets via pandas.

>_ REQUIRED DEPENDENCIES

```
uv add langchain langchain-community langchain-experimental \
      pandas openpyxl sqlalchemy transformers
```



Step 1: Giving the LLM the Database's Shape

Before generating SQL, the LLM needs table and column context

```
from langchain_community.utilities import SQLDatabase

db = SQLDatabase.from_uri("sqlite:///company_data.db")

print(db.get_table_info())
# Outputs table names, column names, types, and sample rows per
table
```

COMPACT SCHEMA DESCRIPTION

`get_table_info()` gives the LLM a compact, structured description of your database, which gets inserted into the prompt instead of retrieved document chunks.

WHY SAMPLE ROWS MATTER

Including a few sample rows (not just column names) significantly improves query accuracy, since it shows the LLM what the actual data values and formats look like (e.g., date format, status codes).



Step 2: Turning a Question into SQL

Sending schema and question to synthesize an exact SQL query

```
from langchain_core.prompts import ChatPromptTemplate

sql_prompt = ChatPromptTemplate.from_template("""
Given this database schema, write a SQL query to answer the question.
Return ONLY the SQL query, no explanation.

Schema:
{schema}

Question: {question}
SQL Query: """)

def generate_sql(question, db, llm):
    query = llm.invoke(sql_prompt.format(schema=db.get_table_info(),
question=question))
    return query.content.strip().strip("`sql").strip("`")

sql_query = generate_sql("What was total revenue last quarter?", db, llm)
print(sql_query)  # SELECT SUM(revenue) FROM sales WHERE quarter = 'Q3';
```

! **Strict SQL output:** "Return ONLY the SQL query" matters. LLMs love to add explanation text around code, which breaks naive execution if not stripped out.

🔄 **Executable query rewriting:** This is the structured-data equivalent of "query rewriting" in standard RAG, except instead of rewriting for vector search, you generate an executable query.



Step 3: Actually Running the Query, **Safely**

Enforcing read-only execution and exception handling

```
DANGEROUS_KEYWORDS = ["DROP", "DELETE", "UPDATE", "INSERT",
                        "ALTER", "TRUNCATE"]
```

```
def safe_execute(sql_query, db):
    if any(keyword in sql_query.upper() for keyword in
DANGEROUS_KEYWORDS):
        return "BLOCKED: Only read (SELECT) queries are
permitted."
    try:
        result = db.run(sql_query)
        return result
    except Exception as e:
        return f"SQL_ERROR: {str(e)}"

result = safe_execute(sql_query, db)
```

🔒 MANDATORY READ-ONLY ROLES

Never let an LLM-generated query run with write permissions. Connect using a database role that only has SELECT access at the database level; don't rely on keyword-blocking alone (it can be bypassed).

🛡️ DEFENSE IN DEPTH

Keyword blocking here is a second layer of defense, not your primary one. Actual database-level read-only permissions are what you must depend on in production.



Step 1: Getting Spreadsheet Data into a Queryable Form

Loading tabular files into pandas DataFrames

```
import pandas as pd

# CSV
df = pd.read_csv("sales_data.csv")

# Excel (can specify a sheet name)
df = pd.read_excel("quarterly_report.xlsx", sheet_name="Q3 Sales")

print(df.dtypes)    # check column types before querying
print(df.head())    # sanity check the actual data
```

Q WHY SANITY CHECKING DTYPES MATTERS

Always inspect dtypes and a few sample rows before building anything on top. A common failure is treating a numeric column that pandas loaded as text (e.g., because of a stray currency symbol) and getting silently wrong aggregation results.



In-memory structure: The DataFrame is the in-memory structure that LangChain's DataFrame agent will operate on directly.





Step 2: Letting the LLM Write & Run Pandas Code

Generating and executing Python REPL commands against your DataFrame

```
from langchain_experimental.agents import create_pandas_dataframe_agent
from langchain_openai import ChatOpenAI # or any chat model

agent = create_pandas_dataframe_agent(
    llm=llm,
    df=df,
    verbose=True,
    allow_dangerous_code=True # required: see safety note below
)

response = agent.invoke("What was the average order value in March?")
print(response["output"])
```



Under the hood: This agent generates Python and pandas code (e.g., `df[df['month'] == 'March']['order_value'].mean()`) and executes it in a Python REPL. Conceptually identical to text-to-SQL, just for DataFrames.



Sandbox required: `allow_dangerous_code=True` is required because this agent executes arbitrary generated Python. Only use it in a sandboxed environment, and never expose directly to untrusted user input without isolation.



Handling Many Tables: Schema Retrieval

What to do when you have too many tables or files to fit in one prompt

```
table_descriptions = {
    "sales": "Order-level sales transactions: date, product, revenue,
region.",
    "customers": "Customer profiles: signup date, plan tier, region.",
    "support_tickets": "Support tickets: status, priority, resolution
time.",
}

# Embed each description, store in a small vector index
from sentence_transformers import SentenceTransformer
embedder = SentenceTransformer("BAAI/bge-small-en-v1.5")
table_embeddings = {name: embedder.encode(desc) for name, desc in
table_descriptions.items()}

def find_relevant_tables(question, table_embeddings, top_k=2):
    q_vec = embedder.encode(question)
    scores = {name: (vec @ q_vec) for name, vec in
table_embeddings.items()}
    return sorted(scores, key=scores.get, reverse=True)[:top_k]

relevant_tables = find_relevant_tables("Status of ticket #4521?",
table_embeddings)
# → ["support_tickets"]
```

🔗 SCHEMA LINKING

Instead of dumping your entire database schema into every prompt (expensive and confusing for the model at scale), retrieve just the tables relevant to the question first, then generate SQL only against that narrowed schema.



Query Validation & Self-Correction

Feeding execution errors back to the LLM to self-correct automatically

```
def generate_and_execute_with_retry(question, db, llm, max_retries=2):
    schema = db.get_table_info()
    query = generate_sql(question, db, llm)

    for attempt in range(max_retries):
        result = safe_execute(query, db)
        if not str(result).startswith("SQL_ERROR") and not
str(result).startswith("BLOCKED"):
            return result

        # Feed the error back and ask for a corrected query
        retry_prompt = f"""The following SQL query failed: {query}
Error: {result}
Schema: {schema}
Question: {question}
Write a corrected SQL query. Return ONLY the SQL."""
        query = llm.invoke(retry_prompt).content.strip()

    return "Could not generate a working query after retries."
```

THE GRADE & RETRY LOOP

This mirrors the "grade -> retry" loop from agentic RAG, except here, the "grading" is simply whether the query executed successfully. Always cap retries; a malformed query that keeps failing the same way will otherwise loop forever.



Turning Raw Data into a Natural Answer

Translating raw tuples and DataFrames into conversational responses

```
format_prompt = """Given the question and the raw query result below,  
write a clear, natural language answer. Be concise and precise with  
numbers.
```

```
Question: {question}  
Raw result: {result}  
Answer: ""
```

```
def format_answer(question, result, llm):  
    return llm.invoke(format_prompt.format(question=question,  
result=result)).content
```

```
final_answer = format_answer(  
    "What was total revenue last quarter?", "[('487213.50',)]", llm  
)  
# → "Total revenue last quarter was $487,213.50."
```

✍ WHY FORMATTING MATTERS

Raw query output `[('487213.50',)]` is technically correct but unusable directly. This formatting step is what actually makes structured RAG feel like a conversational assistant rather than a database console.





Combining Structured & Unstructured Sources

Real systems need both: routing dynamically between paths

```
router_prompt = """Classify this question as: SQL_DATA, DOCUMENT_SEARCH,  
or BOTH.
```

```
Question: {question}  
Classification: ""
```

```
def route_question(question, llm):  
    result = llm.invoke(router_prompt.format(question=question))  
    return result.content.strip()
```

```
route = route_question("Refund policy vs refunds processed last month?",  
llm)  
# → "BOTH" → run both SQL path & document-RAG path, then combine  
answers
```



MULTI-TOOL ROUTING PATTERN

A router decides whether a question needs the SQL or DataFrame path, the standard document-RAG path, or both, since real business questions often span both ("What's our refund policy, and how many refunds did we process last month?").



Structured data is simply another "tool" the agent chooses between, alongside vector store retrieval and web search.



Measuring Whether This is **Actually Working**

Key performance indicators specific to structured retrieval

✓ **Execution accuracy:** Did the generated query run without error? (the most basic check)

🎯 **Result accuracy / exact match:** Does the query's output match the correct expected value for a labeled test set of questions?

⚖️ **Semantic equivalence:** Two different SQL queries can both be "correct" (different joins, same result); grade based on whether results match, not whether the SQL text matches.

👤 **Schema linking accuracy:** For large databases, did the system select the right tables and columns before generating the query?

```
def evaluate_sql_accuracy(test_questions, expected_results, db, llm):
    correct = 0
    for question, expected in zip(test_questions, expected_results):
        query = generate_sql(question, db, llm)
        result = safe_execute(query, db)
        if str(result).strip() == str(expected).strip():
            correct += 1
    return correct / len(test_questions)
```



Common Pitfalls in Structured RAG

Mistakes that show up constantly when deploying to production

WRITE ACCESS "JUST IN CASE"

Always connect with a read-only database role; never rely solely on prompt instructions or keyword filtering to prevent destructive queries.

DUMPING THE ENTIRE SCHEMA

Wastes tokens and confuses the model once you have more than a handful of tables; use schema retrieval (Slide 10) instead.

NO SAMPLE DATA SHOWN

The LLM can't infer date formats, status code meanings, or units without seeing actual example values alongside column names.

SKIPPING THE FORMATTING STEP

Returning a raw tuple or DataFrame as the "answer" instead of natural language breaks conversational UX.

NO RETRY / ERROR-HANDLING LOOP

A single failed query becomes a dead end instead of a chance to self-correct and recover gracefully.

GRADING BY EXACT SQL TEXT MATCH

Grade by result correctness, since multiple valid queries can produce the exact same right answer.

